

DarwinTrade: Hierarchical Self-Evolution for Multi-Agent Portfolio Trading

Anonymous submission

Abstract

Large language models (LLMs) can integrate prices, news, fundamentals, and other financial evidence for portfolio decisions. However, existing LLM-based trading systems compress every policy adaptation into a single LLM action, so a return outcome cannot be traced back to the specific signal or bounded component that produced it. Modeled after the specialist structure of a real investment firm, we introduce DarwinTrade, a multi-agent trading system that instantiates *hierarchical self-evolution*: policy adaptation is decomposed into three state-disjoint loops that self-evolve on their own clocks, over a deterministic executor that the LLM never edits. An analyst loop updates per-role reliability from delayed cross-sectional information coefficients, a tactical loop installs 1-3 bar risk guards that expire automatically, and a strategic loop commits or reverts LLM-authored allocator patches against a logged rollback trigger. Every input consumed on trading day t is drawn strictly from information available before t , and realized outcomes update future states only after a one-bar delay, so no future information can leak into the day- t decision. On a fixed 20-stock universe with transaction costs, DarwinTrade achieves cumulative returns of 12.1% and 10.9% with Sharpe ratios of 2.07 and 2.56 in the StockBench Rising-Market Window and a later Down-Market Window, ranking first on cumulative return and information ratio in both windows and on Sharpe and Sortino in the Rising-Market Window.

Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in synthesizing prices, news, fundamentals, and other financial evidence for portfolio decisions (Zhang et al. 2024; Liu and Dang 2025a; Rashid et al. 2026). Unlike one-shot question answering, sequential portfolio management is a long-horizon decision task that jointly determines returns through role-conditioned signal aggregation, short-term risk control, and long-lived allocator settings. However, the market itself is non-stationary at multiple time scales simultaneously, so any static or single-update trading policy inevitably becomes stale on some dimension.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

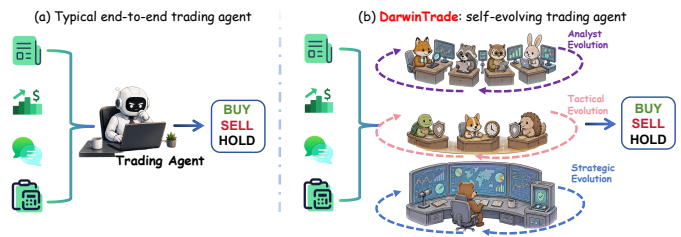


Figure 1: DarwinTrade separates analyst reliability, short-lived tactical guards, and validated allocator changes as three state-disjoint self-evolution loops over a deterministic allocator that the LLM never edits.

A real-world investment firm does not run a portfolio through a single trader who regenerates every decision from scratch each day. Instead, research analysts continually re-weight their views from realized outcomes, risk officers impose short-lived constraints when acute events break the risk budget, and a portfolio committee revises the long-term investment mandate only when a persistent pattern justifies it and reverses that change when it fails. Yet existing LLM-based trading systems collapse all three of these functions into a monolithic prompt-and-reflect loop, treating each trading day as an isolated model-generated action without a persistent record of what evidence has been vetted or how past interventions actually panned out. Ideally, an LLM-based trading system should operate like a real investment firm, weighting its research sources by their realized track records, managing short-term risk without letting a single event override long-term policy, and revising its long-term strategy only when persistently justified. In this way, adaptation is decomposed across the three levels at which change happens in a real trading operation, with each level operating on its own rhythm and every change inspectable and reversible.

Existing approaches to adapting LLM-based trading systems broadly fall into *reflection-based agents* and *program-editing agents*, neither of which delivers all three qualities above. Reflection-based agents such as FinMem, FinAgent, TradingAgents, and FinCon (Yu et al. 2023; Zhang et al. 2024; Xiao et al. 2024; Yu et al. 2024) revise later decisions from stored outcomes or role-specific feedback within a single model-generated action. As a result, bar-level noise

can rewrite regime-level analyst weights, temporary risk responses can silently become permanent, and allocator settings drift without a rollback check, so none of the three qualities above holds. More recently, program-editing agents such as QuantAgent, TradingGroup, and 3S-Trader (Wang et al. 2024; Tian, Salim, and Xue 2025; Chen et al. 2025a) adapt generated programs, styles, or stock-selection strategies from realized feedback. Although these systems demonstrate stronger structural adaptation, they let the LLM rewrite executable behavior directly without an explicit rollback trigger, so a harmful strategy edit can silently persist and the third quality is only partially and irreversibly met. In essence, both lines share a monolithic adaptation loop that collapses the three specialist functions of a real trading firm into one model-generated action, and cannot jointly deliver the three qualities identified above.

In this paper, we aim to advance LLM-based trading systems from monolithic reflection to *hierarchical self-evolution* that jointly delivers the three qualities identified above. To this end, we introduce DarwinTrade, a self-evolving *multi-agent trading system* that decomposes policy adaptation into three state-disjoint loops, namely analyst evolution, tactical evolution, and strategic evolution, over a deterministic executor that the LLM never edits. As illustrated in Figure 1, each loop owns one component of the policy and is triggered on its own rhythm, matched to the rate at which that component becomes stale. Specifically, analyst evolution updates role reliability from delayed next-open returns via a positive-IC gate with sample-size shrinkage, so bar-level noise drops a role instead of flipping it. Tactical evolution installs a 1-3 bar guard in response to a single adverse bar and expires it automatically, with a logged post-install return that prevents a hurting guard family from re-arming. Strategic evolution commits an LLM-authored patch bounded to a validated parameter set if a logged loss threshold is not breached within a horizon of H bars, and reverts the patch the moment that threshold is breached. Through this hierarchical decomposition, every policy change is traceable to a specific loop and a realized outcome, turning adaptation from a hidden reflection loop into an accountable, bounded process.

In summary, our key contributions are three-fold.

- **Hierarchical Self-Evolution.** To the best of our knowledge, DarwinTrade is one of the first multi-agent LLM trading systems to instantiate *hierarchical self-evolution*, decomposing policy adaptation into three state-disjoint loops aligned with the specialist structure of a real investment firm and running over a deterministic executor that the LLM never edits.
- **Bounded Instantiation at Three Levels.** We instantiate the three loops with a regime-conditioned IC-shrinkage capsule (analyst), an expiring track-record-gated guard (tactical), and an LLM-authored patch with a logged rollback trigger (strategic), all logged as separate operative states so every policy change remains traceable to a specific loop.
- **Strong Performance.** On a fixed 20-stock universe with transaction costs, DarwinTrade achieves 12.1% and 10.9% cumulative returns and 2.07 and 2.56 Sharpe ratios in the StockBench Rising-Market and Down-Market Windows,

and disabling all three loops halves Sharpe to 0.94 and deepens drawdown to -15.2% in the Rising-Market Window.

Related Work

Existing LLM-based trading systems broadly fall into two lines: *reflection-based agents* with role specialization and memory, and *program-editing agents* that adapt executable behavior.

Reflection-Based Agents. FinMem, FinAgent, TradingAgents, and FinCon revise later decisions from stored outcomes or role-specific feedback (Yu et al. 2023; Zhang et al. 2024; Xiao et al. 2024; Yu et al. 2024), while more recent systems make position sizing, downside risk, and specialist evidence more explicit (Liu and Dang 2025a,b; Rashid et al. 2026). Although these systems demonstrate stronger reasoning, they generally fuse research, portfolio construction, and adaptation in one model-generated action and do not maintain analyst reliability, expiring tactical interventions, and validated allocator changes as three separately logged operative states. Recent benchmarks and audits reinforce this concern (Chen et al. 2025b; Yu, Li, and You 2025; Fan et al. 2025; Yao and Zheng 2026; Qu and Chen 2026; Li et al. 2026; Duan et al. 2026), showing that static reasoning capability does not transfer to sequential portfolio performance without point-in-time controls and intermediate decision traces.

Program-Editing Agents. QuantAgent, TradingGroup, and 3S-Trader adapt generated programs, styles, or stock-selection strategies from realized feedback (Wang et al. 2024; Tian, Salim, and Xue 2025; Chen et al. 2025a). These systems admit that persistent problems require persistent policy changes; however, they still let the LLM rewrite executable behavior directly and lack explicit rollback when a proposed edit fails to justify itself on subsequent outcomes, so a harmful edit can quietly become permanent policy. Related studies also highlight look-ahead leakage, passive exposure, and regime-dependent performance as confounds (Glasserman and Lin 2024; Fu 2025; Zhu et al. 2026; Jeon and Lee 2026) that a bounded editing surface would need to address. In parallel, generic agent frameworks such as ReAct and Reflexion (Yao et al. 2022; Shinn et al. 2023) update behavior from environmental feedback, and classical portfolio construction such as mean-variance optimization, active management, and hierarchical risk parity (Markowitz 1952; Grinold and Kahn 2000; Lopez de Prado 2016) separates forecasts from risk-controlled allocation, but neither line addresses the outcome-driven evolution of specialist components inside a bounded policy scope.

In this work, we focus on the auditability and reversibility of policy adaptation, with the goal of turning realized outcomes into policy changes that can be independently attributed to analyst, tactical, or strategic components. Unlike previous approaches that either fuse research and execution in one model-generated action or let the LLM rewrite executable behavior directly, DarwinTrade adopts a more deliberate design by decoupling three hierarchical loops of self-evolution from a deterministic allocator and execution, namely regime-conditioned analyst reliability learned from

System	Analyst		Tactical		Strategic	
	Present	Evolving	Present	Evolving	Present	Evolving
FinMem	✗	✗	✓	✓	✗	✗
FinAgent	✗	✗	✓	✓	✗	✗
TradingAgents	✓	✗	✓	✓	✓	✗
FinCon	✓	✓	✓	✓	✗	✗
QuantAgent	✓	✗	✗	✗	✓	✓
DarwinTrade	✓	✓	✓	✓	✓	✓

Table 1: Only DarwinTrade activates and evolves all three operative layers (analyst, tactical, strategic), whereas prior LLM trading systems either omit a layer or leave it non-evolving. “Present” marks whether the layer exists at all (✓ yes / ✗ no); “Evolving” marks whether realized feedback updates the layer’s operative state. Analyst evolution changes role influence, tactical evolution changes near-term decisions, and strategic evolution changes executable portfolio-control state.

delayed next-open returns, expiring tactical guards, and validated strategic changes with automatic rollback. Table 1 compares these update layers in representative LLM-based trading systems.

Method

Modeled after the specialist structure of a real investment firm, DarwinTrade instantiates hierarchical self-evolution as three state-disjoint loops (analyst, tactical, and strategic) running over a deterministic executor that the LLM never edits. As shown in Figure 2, on trading day t four LLM analysts process market, news, fundamental, and social evidence dated strictly before t . Their scores are fused under regime-conditioned reliability, filtered by any active tactical guards, and converted into target weights by a deterministic allocator. DarwinTrade freezes weights before the opening auction and uses the day- t open only as the fill price, and realized open-to-open returns update future states only after a one-bar delay, so no future information can leak into the day- t decision. We call one completed decision bar with its next-open outcome an *episode*, the unit on which the strategic review cadence is counted. Appendix A states the complete per-bar and periodic control loop as pseudocode, and Appendix B reproduces every LLM prompt and output schema.

Analyst Evolution

Forward Process. Because forecast and trading skill vary materially across regimes (Zhang et al. 2025; Jeon and Lee 2026), DarwinTrade first identifies the regime and then performs regime-conditioned role fusion. The LLM module f_{regime} classifies the market as bull, bear, or volatile from SPY’s 5- and 20-bar trailing returns dated before t , where “volatile” labels a *trend-disagreement* state in which the short- and long-horizon trends carry opposite signs. For each asset i , four role-specific LLM modules f_k ($k \in \mathcal{K}$) process

market, news, fundamentals, and social evidence in parallel (Xiao et al. 2024; Rashid et al. 2026), each returning a directional score $x_{t,i,k} \in [-1, 1]$ and confidence $c_{t,i,k} \in [0, 1]$. Given regime-conditioned role reliability $u_{\rho,t,k,t}$, we compute analyst influence $\alpha_{t,i,k}$ and aggregate score $s_{t,i}$ as

$$\alpha_{t,i,k} = u_{\rho,t,k,t} c_{t,i,k}, \quad s_{t,i} = \frac{\sum_{k \in \mathcal{K}} \alpha_{t,i,k} x_{t,i,k}}{\sum_{k \in \mathcal{K}} \alpha_{t,i,k}}, \quad (1)$$

where $u_{\rho,t,k,t}$ is learned from realized outcomes and $c_{t,i,k}$ is the analyst’s self-reported confidence. Scores above 0.15 produce buy candidates and below -0.15 sell candidates. Analyst prompts and their shared output schema are in Appendix B.1.

Self-Evolution Loop. DarwinTrade maintains one evaluation capsule $\mathcal{C}_{\rho,k,t}$ per regime–role pair (ρ, k) , storing the latest 200 tuples $(x_{t,i,k}, c_{t,i,k}, y_{t+1,i}, t)$. After removing the cross-sectional mean of each trade date from both scores and returns, we compute their Pearson correlation $\text{IC}_{\rho,k,t}$, following the use of information coefficients in active management (Grinold and Kahn 2000). The role skill used on the next bar is

$$u_{\rho,k,t+1} = f_{\text{analyst-evolving}}(\mathcal{C}_{\rho,k,t}) = \begin{cases} u_0, & n < 20, \\ \max(0, \text{IC}) \frac{n}{n+20}, & n \geq 20, \end{cases} \quad (2)$$

where $n = n_{\rho,k,t}$ is the capsule size, $\text{IC} = \text{IC}_{\rho,k,t}$ is the demeaned Pearson correlation between $\{x_{t,i,k}\}$ and $\{y_{t+1,i}\}$, $u_0 = 0.01$ is a cold-start value, and $n/(n+20)$ is a sample-size shrinkage that damps IC estimates supported by few observations. Because IC is truncated at zero, a warm role with negative IC is removed instead of reversed, so bar-level noise cannot flip role semantics. Since $y_{t+1,i}$ is unavailable at decision time t , the update is strictly one bar delayed.

Tactical Evolution

Forward Process. Short-lived events may require immediate risk control that should not permanently change the allocation policy (Liu and Dang 2025a,b). To this end, DarwinTrade maintains a tactical state $\mathcal{U}_t^T$ with an avoid set, a reduce-only set, a portfolio haircut in $[0.4, 1.0]$, and a time-to-live. Every guard expires after one to three trading days, so a tactical response constrains the next decision without silently becoming permanent policy.

Self-Evolution Loop. After the next-open portfolio return y_t^{pf} becomes available, the tactical loop composes a reflection over the previous decision trace and realized return with an install-with-TTL step over the guard state:

$$\mathcal{U}_{t+1}^T = f_{\text{tactical-evolving}}(\tau_{t-1}, y_t^{\text{pf}}, \mathcal{U}_t^T), \quad (3)$$

where τ_{t-1} is the stored decision trace of the preceding episode, y_t^{pf} is the realized next-open portfolio return, and $\mathcal{U}_t^T$ is the current tactical guard state. Internally, $f_{\text{tactical-evolving}}$ produces a guard action that may install an avoid, reduce-only, or haircut constraint, or remain empty. The installation respects the 1–3 bar time-to-live and logs its anchor NAV and post-install return at expiry, so a hurting guard family cannot keep re-arming. Tactical prompts and schema appear in Appendix B.2.

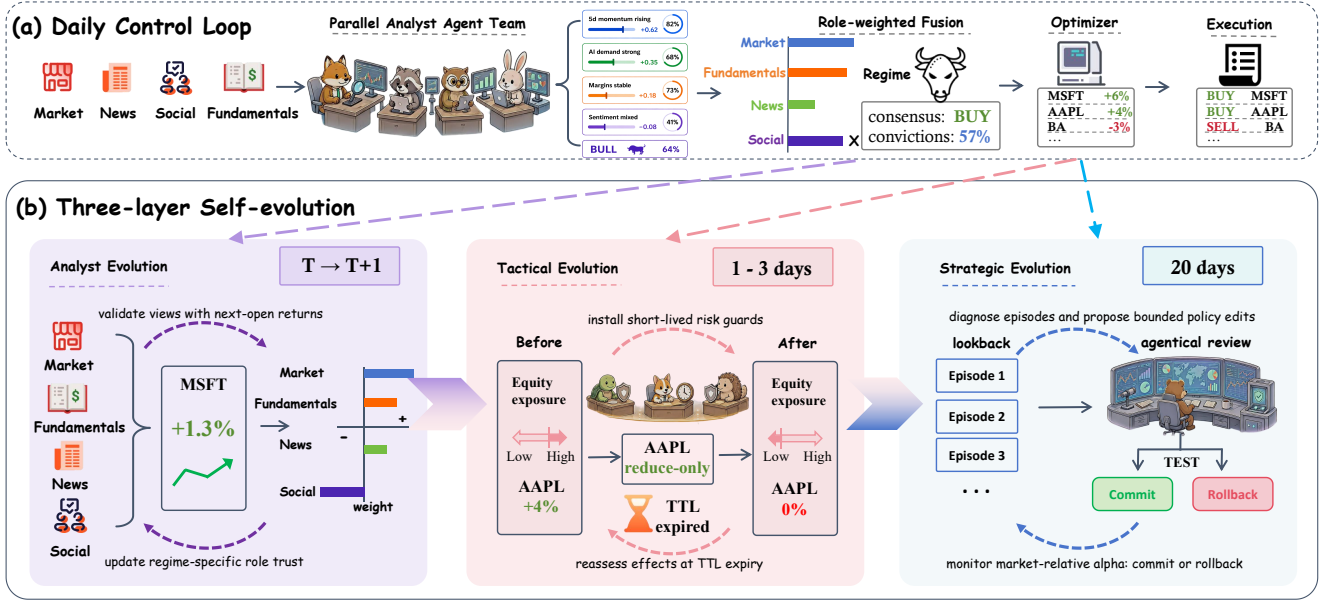


Figure 2: DarwinTrade architecture: a daily control loop over a deterministic executor, driven by three self-evolution loops (analyst, tactical, strategic) that update disjoint policy states on their own clocks.

Strategic Evolution

Forward Process. Persistent allocation problems require changing the long-lived policy (Tian, Salim, and Xue 2025; Chen et al. 2025a). DarwinTrade parameterizes the deterministic allocator by the strategic policy π_t . Given the guarded signals $\tilde{S}_t$, current portfolio P_t , and historical returns $\mathcal{R}_{<t}$, the allocator produces

$$w_t = \text{Allocate}(\tilde{S}_t, P_t, \mathcal{R}_{<t}; \pi_t), \quad (4)$$

$$\text{s.t. } |w_{t,i}| \leq w_{\max,t}, \quad \sum_i |w_{t,i}| \leq G_t,$$

where G_t is the gross-exposure budget, $w_{\max,t}$ the per-name cap, and $\text{Allocate}(\cdot)$ a pure deterministic function of its inputs and π_t . The policy selects one of nine allocator modes spanning classical mean-variance, active-allocation, and hierarchical-risk-parity families (Markowitz 1952; Grinold and Kahn 2000; Lopez de Prado 2016); the full mode list is in Appendix D.

Self-Evolution Loop. A single episode is insufficient evidence for changing a long-lived policy (Rashid et al. 2026; Qu and Chen 2026), so DarwinTrade forms a twenty-day history $\mathcal{H}_j^S$ after every three completed episodes for review cycle j . The strategic loop reviews this history, authors a candidate patch bounded to Ω , and either commits the patch or rolls back, yielding the next policy:

$$\pi_{j+1} = f_{\text{strategic-evolving}}(\mathcal{H}_j^S, \pi_j, \Omega), \quad (5)$$

where $\mathcal{H}_j^S$ is the twenty-day history for review cycle j , π_j is the current strategic policy, and Ω is the allowed parameter set (nine allocator modes plus seven bounded scalar fields; Appendix B.2). Internally, $f_{\text{strategic-evolving}}$ proposes a patch $(\Delta\pi_j, H_j, \varepsilon_j, m_j)$ over Ω and validates it. Letting

$g_{m_j}(t_j, t)$ be the monitored performance from installation bar t_j under mode m_j , the loop restores π_j at the first bar with $g_{m_j}(t_j, t) < -\varepsilon_j$ and commits the proposal if no breach occurs by $t_j + H_j$. Only one proposal is evaluated at a time, so strategic evolution changes only validated allocator parameters through a logged commit-or-rollback loop and cannot edit the execution engine.

Experiments

Experimental Setup

Benchmark. We use a fixed daily U.S. equity universe of 20 large-cap stocks (full list in Appendix E), unchanged across both windows and not reselected using period-specific information. Each evaluation starts with USD 100,000 and assumes full fills, a 1 bp commission, and 2 bps slippage. Relative metrics use SPY as the benchmark where available. Full backtest configuration, the LLM backbone and decoding settings, and baseline hyperparameters are reported in Appendix C. The primary evaluation follows the Rising-Market Window distributed with StockBench (Chen et al. 2025b), spanning 2025-03-03 to 2025-06-30, while the chronologically later Down-Market Window spans 2025-12-01 to 2026-03-31, during which SPY falls -4.6% . The Down-Market Window changes the market environment while keeping the universe, controller code, policy space, execution assumptions, and transaction-cost model fixed, and therefore serves as an independently initialized evaluation under a different market environment, not a separately tuned setting. The two windows are evaluated independently. All three evolution states start empty and adapt only from feedback observed within that window. Thus, “the same controller” refers to frozen code and policy bounds whose evolution restarts for

each evaluation. Each window contains 83 decision bars and 82 return intervals.

Data Controls. Every input used at day t , including price bars, news, fundamentals, and social data, is admitted only when its timestamp is strictly before t , and prices are adjusted for splits and dividends following standard vendor conventions. Because the allocator consumes only relative cross-sectional signals, the ex-post nature of these adjustments does not alter the ordering used at any decision bar. LLM prompts contain only pre- t evidence and never receive the day- t open or any subsequent information, so no future information can enter the decision path. Analyst reliability is learned only from realized post-decision returns, and allocation remains in the deterministic controller. These controls follow recent recommendations to make point-in-time admission, execution timing, and intermediate decision traces explicit (Yao and Zheng 2026; Qu and Chen 2026; Li et al. 2026).

Baselines. In the Rising-Market Window, we evaluate the complete 2^3 factorial over analyst, tactical, and strategic evolution: the analyst ablation disables online IC updates, whereas the no-evolution variant disables all three feedback loops. Beyond these ablations, we compare DarwinTrade against buy-and-hold, moving-average crossover, Bollinger bands, ATR bands, ARIMA (Box et al. 2015), XG-Boost (Chen and Guestrin 2016), TradingAgents (Xiao et al. 2024), and FinAgent (Zhang et al. 2024). All methods use the same universe, evaluation windows, and transaction-cost model.

Metrics and Statistical Tests. We report cumulative return, maximum drawdown, annualized Sharpe ratio, annualized Sortino ratio, annualized information ratio against SPY, and trade count. To compare paired daily returns while accounting for serial dependence, we use a moving-block bootstrap (Künsch 1989) with block length 5 and 5000 resamples, reporting the paired excess-return estimate $\Delta_\Sigma = 100 \sum_t (r_t^{\text{ours}} - r_t^{\text{comp}})$ in percentage points and P_+ , the bootstrap probability that Δ_Σ is positive.

Main Results

Cross-Regime Portfolio Capability. We evaluate DarwinTrade on the StockBench Rising-Market and later Down-Market Windows under the same controller code, policy bounds, and transaction-cost model. We compare DarwinTrade with rule-based (5), machine-learning (2), classical-quant (4), and LLM-agent (2) baselines under a common 20-stock universe. As shown in Table 2, DarwinTrade achieves cumulative returns of 12.1% and 10.9% with Sharpe ratios of 2.07 and 2.56 in the rising and down windows, outperforming the best per-window baseline return by 8.7 and 4.3 percentage points respectively and ranking first on cumulative return and information ratio in both windows, as well as on Sharpe and Sortino in the Rising-Market Window. In the Down-Market Window a classical minimum-variance construction over the same allocator (MVQ + min-variance, 7.7%/Sharpe 2.76/Sortino 4.71) attains higher risk-adjusted metrics on a lower cumulative return, so no single method dominates every metric. Unlike fixed rule-based strategies whose ranking flips between windows, e.g. Bollinger bands lead the rising

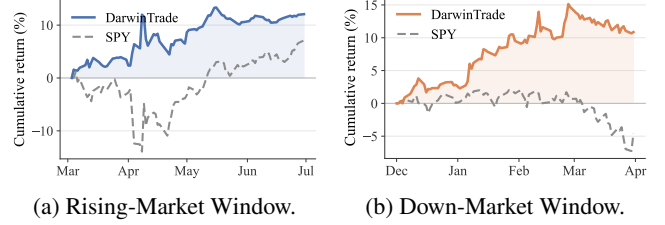


Figure 3: DarwinTrade outperforms SPY in both the Rising-Market and Down-Market Windows under the common 20-stock universe and transaction-cost model, achieving positive cumulative return in a -4.6% down window without re-tuning the controller code or policy space.

window (3.4%) but lose 1.3% in the down window, while WMA reverses (-6.5% then $+6.6\%$), DarwinTrade’s cross-window return spread is only 1.2 percentage points against the best baseline’s 13.1. More importantly, feeding a non-LLM momentum-value-quality composite into the same allocator (the four MVQ rows) further isolates the analyst layer: every construction loses in the rising window (-4.7% to -6.3%) and only modestly profits in the down window, so the shared allocator alone does not explain DarwinTrade’s cross-window gains. Overall, DarwinTrade’s results are consistent with adaptation inside a specified policy space, not a different strategy per regime.

Comparison with External LLM Agents. Zooming in on LLM-agent baselines, DarwinTrade also outperforms TradingAgents ($-0.0\%/ -2.0\%$) and FinAgent ($-2.4\%/ -1.3\%$) in both windows by more than 12 and 12 percentage points respectively. The distinction is not the number of agents: end-to-end reflection rewrites subsequent model-generated behavior in one opaque update, whereas DarwinTrade records analyst reliability, temporary risk state, and validated allocator state separately before deterministic execution, so no reflection can rewrite order logic. This aligns with recent evidence that reasoning strength does not automatically yield stable sequential portfolio performance (Chen et al. 2025b; Yu, Li, and You 2025). Overall, DarwinTrade’s strong results against external LLM agents highlight its advanced capability to convert LLM research into risk-adjusted sequential performance. Figure 3 plots the NAV trajectories across both windows.

Analyses

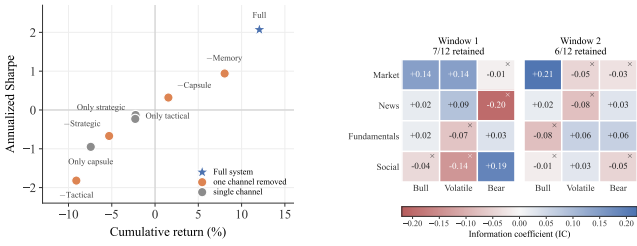
Ablation Study on Hierarchical Self-Evolution

To separate each evolution layer’s contribution from cross-layer coordination, we run the complete 2^3 factorial in the Rising-Market Window and inspect regime-conditioned analyst selection alongside logged tactical and strategic interventions (Figure 4, Table 3). Table 3 reports four ablation contrasts, and the full eight-cell factorial is in Appendix F.

As reported in Table 3, the three loops carry different functions in DarwinTrade. In particular, removing tactical evolution causes the largest single-layer loss (-9.1% return, -1.82 Sharpe), demonstrating the advantage of the one-to-

Category	Method	Rising-Market Window					Down-Market Window				
		Return	Max drawdown	Sharpe	Sortino	Information ratio	Return	Max drawdown	Sharpe	Sortino	Information ratio
Rule	Buy-and-hold	0.008	-0.152	0.25	0.33	-0.29	-0.044	-0.096	-1.06	-1.60	0.01
	Bollinger bands	0.034	-0.054	0.73	1.35	-0.37	-0.013	-0.028	-1.21	-1.90	0.75
	ATR bands	0.007	-0.067	0.23	0.21	-0.65	0.007	-0.005	1.54	1.42	1.21
	SMA crossover	-0.010	-0.124	-0.07	-0.07	-1.19	0.058	-0.029	1.75	3.39	1.80
	WMA crossover	-0.065	-0.172	-0.86	-0.81	-1.94	0.066	-0.025	2.03	3.75	1.86
Machine learning	ARIMA	-0.036	-0.113	-0.45	-0.58	-1.23	-0.052	-0.075	-1.60	-2.13	-0.17
	XGBoost	-0.044	-0.074	-1.42	-1.46	-1.27	-0.039	-0.057	-1.92	-2.87	0.14
Classical quant	MVQ + equal-weight	-0.052	-0.083	-1.38	-1.79	-1.20	0.039	-0.058	1.35	1.86	1.72
	MVQ + HRP	-0.059	-0.092	-1.50	-1.97	-1.21	0.047	-0.047	1.75	2.72	1.83
	MVQ + min-variance	-0.063	-0.091	-1.62	-2.13	-1.27	0.077	-0.039	2.76	4.71	2.39
	MVQ + max-Sharpe	-0.047	-0.090	-0.99	-1.14	-1.07	0.026	-0.047	0.90	1.43	1.62
LLM agent	TradingAgents	-0.000	-0.048	0.05	0.07	-0.72	-0.020	-0.038	-1.06	-1.92	0.56
	FinAgent	-0.024	-0.065	-0.66	-0.77	-1.16	-0.013	-0.046	-0.57	-0.92	0.74
Ours	DarwinTrade	0.121	-0.067	2.07	3.20	0.18	0.109	-0.039	2.56	4.24	2.94

Table 2: DarwinTrade achieves the highest cumulative return and information ratio in both the StockBench Rising-Market and the later Down-Market Windows, and leads Sharpe and Sortino in the Rising-Market Window. In the Down-Market Window a classical minimum-variance construction over the same allocator attains higher Sharpe and Sortino. *MVQ* denotes a non-LLM cross-sectional momentum–value–quality composite fed into the same deterministic allocator DarwinTrade uses, under four portfolio constructions. Return is cumulative; Sharpe, Sortino, and information ratio are annualized. **Bold** marks the best value on each metric in each window.



(a) Factorial variants in return–Sharpe space.

(b) Regime-conditioned analyst selectivity.

Figure 4: Three-layer self-evolution decomposes DarwinTrade’s gains into distinct roles. (a) In return–Sharpe space, the full system dominates every 2^3 factorial variant in the Rising-Market Window. (b) The positive-IC gate suppresses 5 of 12 warm analyst capsules, with removed capsules concentrating on non-positive mean IC ($\times$ marks a removed capsule).

three-bar guard as the only loop that can immediately reduce a newly harmful exposure. Without strategic evolution, return falls to -5.3% and maximum drawdown reaches -16.3% , since short-lived guards cannot replace a persistent change to optimizer or portfolio size. Removing analyst evolution leaves portfolio controls intact but weakens cross-sectional selection (1.5% return, 0.32 Sharpe). These are ablation-level associations, not additive causal effects: every single-layer-only variant has negative Sharpe, so the layers must operate on one another’s evolving inputs (Liu and Dang 2025a,b). Notably, the static policy still profits, but evolution mainly

improves the path of returns: disabling all three loops yields 8.0% return yet halves Sharpe from 2.07 to 0.94 and deepens maximum drawdown from -6.7% to -15.2% , supporting improved risk-adjusted adaptation, not standalone return per update. Overall, the ablation results support our design of decomposing self-evolution into three independently ablatable hierarchical loops.

Superiority of Analyst Selection

As shown in Figure 4b, analyst evolution is more than generic multi-agent averaging. The positive-IC gate suppresses 5 of 12 warm capsules in the Rising-Market Window and 6 of 12 in the Down-Market Window. Retained capsules have mean ICs of 0.09 and 0.07 , versus -0.09 and -0.05 for removed ones. Selection is role- and regime-specific, so that the news analyst, for instance, is removed on bear-labeled days in the rising window but kept in bull and volatile regimes, allowing a role to be reused where its delayed evidence stays predictive without gaining permanent influence (Zhang et al. 2025; Jeon and Lee 2026). This gating relies on the regime labels: scored against the deterministic trend rule it follows, the classifier reaches 0.86 directional agreement in both windows with errors confined to the adjacent volatile state and essentially no bull/bear sign flips (Appendix E.1). In contrast, the other two layers leave different traces: tactical interventions expire within one to three days, limiting a local error without permanently suppressing a symbol, while strategic evolution edits approved allocator fields and logs the diagnosis, horizon, and rollback threshold, so its moves toward HRP and risk parity remain bounded and reversible. Overall, the analyst-selection results highlight the advantage of regime-

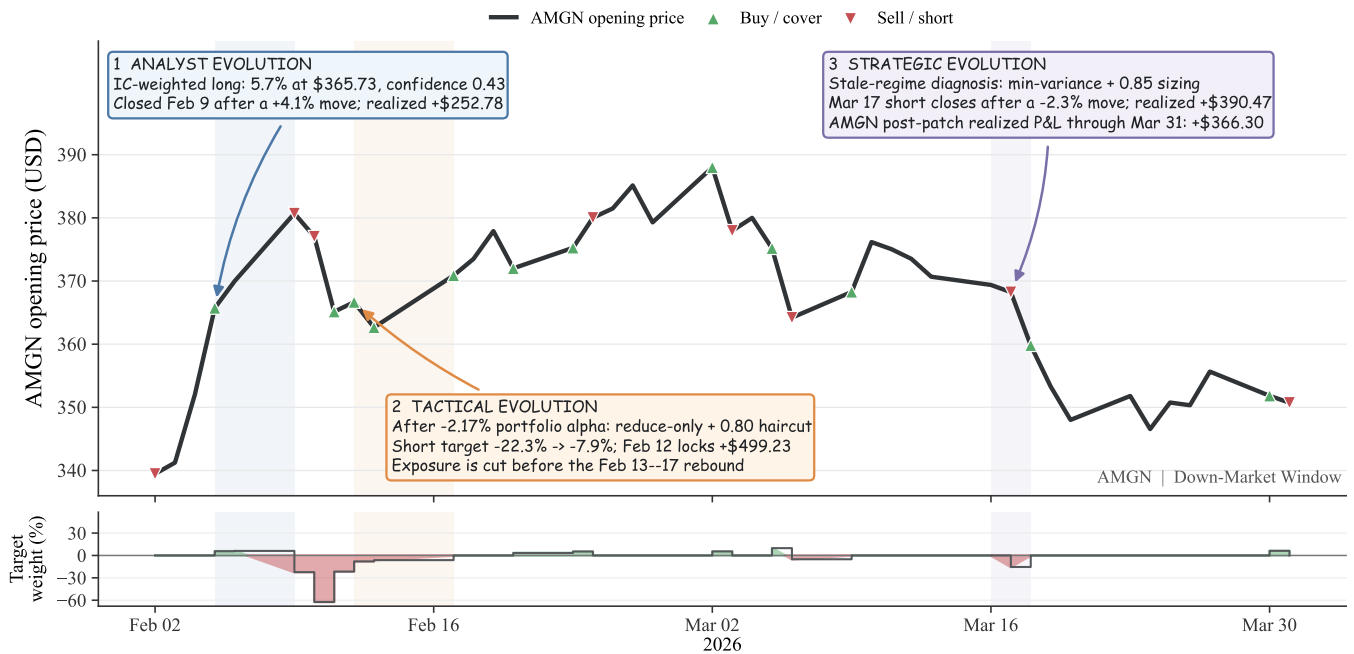


Figure 5: One AMGN symbol carries an inspectable trace of all three timescales acting independently in the Down-Market Window: an analyst entry (1), a two-bar tactical guard (2), and a strategic allocator update (3). The upper panel shows point-in-time opening prices and executions (up: buy/cover; down: sell/short); the lower panel shows the deterministic target weight; numbered callouts report only fields and outcomes recovered from the run logs.

Variant	Return	Max drawdown	Sharpe
Full	0.121	-0.067	2.07
–Tactical	-0.091	-0.148	-1.82
–Strategic	-0.053	-0.163	-0.67
–Analyst	0.015	-0.112	0.32
–All evolution	0.080	-0.152	0.94

Table 3: Removing any of the three self-evolution loops degrades DarwinTrade in the Rising-Market Window, and every single-layer removal turns Sharpe non-positive except the no-analyst variant. –All evolution disables analyst, tactical, and strategic feedback loops jointly.

conditioned reliability over uniform multi-analyst averaging.

Case Study on Hierarchical Evolution

To evaluate whether self-evolution is auditable and not hidden, we trace one symbol to check that each timescale acts independently and leaves an inspectable record. Specifically, we follow Amgen (AMGN) through the Down-Market Window (Figure 5), an auditing example rather than a performance test; the full log excerpt with all logged fields is in Appendix H. As shown in Figure 5, three events act on distinct timescales. At the analyst timescale, IC-weighted fusion opens a 5.7% long at 0.43 confidence and closes it four bars later for +\$252.78. When a negative-alpha day triggers the tactical timescale, AMGN is placed on a two-bar reduce-only list with a 0.80 haircut, which trims the running short before

the mid-February rebound and then expires automatically. At the strategic timescale, a stale-regime diagnosis switches the optimizer to minimum-variance with a 0.85 size multiplier and logs the rollback horizon and threshold, so the subsequent short is sized down and the symbol accumulates +\$3.07k over the window. In particular, each event is attributable to a specific trigger, outcome, and bounded policy component, inspectable on its own instead of as one post-hoc reflection. Overall, the case study demonstrates the advantage of hierarchical decomposition for auditable self-evolution.

Analysis of Exposure and Market Beta

To test whether performance is merely a persistent directional bet (Zhu et al. 2026; Duan et al. 2026), we compute regime-conditioned daily exposure and realized SPY beta. Net exposure moves from -0.22 to 0.31 across regimes in the rising window and stays moderately net long (0.22 – 0.37) in the down window (betas -0.02 and 0.25); this sign-varying exposure contrasts with the regime-reversing baseline rankings (Table 2), arguing against passive SPY direction.

Statistical Robustness

To account for serial dependence with a small effective sample, we perform paired moving-block bootstrap comparisons (5000 resamples) of the summed paired daily excess return Δ_{Σ} (pp, with positive probability P_{+}). The results favor the full system over every rising-window single-layer removal (Tactical: 21.0, 0.94; Strategic: 16.6, 0.91; Analyst: 9.5, 0.88; all-evolution: 2.7, 0.56) and the strongest down-window baseline WMA (4.0, 0.69); a sensitivity analysis

over five design axes (Appendix G) shows that no single alternative uniformly improves on the reported system across both windows.

Conclusion

In this paper, we introduce DarwinTrade, an auditable self-evolving multi-agent trading system that instantiates hierarchical self-evolution as three state-disjoint loops, namely analyst, tactical, and strategic evolution, over a deterministic executor. Under this decomposition, DarwinTrade achieves 12.1% and 10.9% cumulative returns with Sharpe ratios of 2.07 and 2.56 in the StockBench Rising-Market and Down-Market Windows, while every policy change is traceable to a specific loop and a realized outcome.

References

- Box, G. E. P.; Jenkins, G. M.; Reinsel, G. C.; and Ljung, G. M. 2015. *Time Series Analysis: Forecasting and Control*. Wiley, 5 edition.
- Chen, K.; Ahmad, H.; Goel, D.; and Szabo, C. 2025a. 3S-Trader: A Multi-LLM Framework for Adaptive Stock Scoring, Strategy, and Selection in Portfolio Optimization. arXiv:2510.17393.
- Chen, T.; and Guestrin, C. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- Chen, Y.; Yao, Z.; Liu, Y.; Ye, J.; Yu, J.; Hou, L.; and Li, J. 2025b. StockBench: Can LLM Agents Trade Stocks Profitably In Real-world Markets? arXiv:2510.02209.
- Duan, Q.; Li, C.; Wang, C.; Zhang, F.; Wang, M.; Miao, D.; Ma, P.; Yan, J.; Chen, L.; Liu, S.; Nakov, P.; Luo, Y.; and Tang, N. 2026. Can Agentic Trading Systems Pay for Their Own Intelligence? arXiv:2607.10286.
- Fan, T.; Yang, Y.; Jiang, Y.; Zhang, Y.; Chen, Y.; and Huang, C. 2025. AI-Trader: Benchmarking Autonomous Agents in Real-Time Financial Markets. arXiv:2512.10971.
- Fu, W. 2025. The New Quant: A Survey of Large Language Models in Financial Prediction and Trading. arXiv:2510.05533.
- Glasserman, P.; and Lin, C. 2024. Assessing Look-Ahead Bias in Stock Return Predictions Generated by GPT Sentiment Analysis. *The Journal of Financial Data Science*, 6(1): 25–42.
- Grinold, R. C.; and Kahn, R. N. 2000. *Active Portfolio Management: A Quantitative Approach for Producing Superior Returns and Controlling Risk*. McGraw-Hill, 2 edition.
- Jeon, J.; and Lee, H. 2026. Can Blindfolded LLMs Still Trade? An Anonymization-First Framework for Portfolio Optimization. arXiv:2603.17692.
- Künsch, H. R. 1989. The Jackknife and the Bootstrap for General Stationary Observations. *The Annals of Statistics*, 17(3): 1217–1241.
- Li, C.; Ma, P.; Duan, Q.; Lin, Z.; and Wu, P. 2026. NextFund: A Unified Performance Tracking Platform for Agentic Portfolio Management. arXiv:2607.11141.
- Liu, B.; and Dang, R. 2025a. FinPos: A Position-Aware Trading Agent System for Real Financial Markets. arXiv:2510.27251.
- Liu, B.; and Dang, R. 2025b. FINRS: A Risk-Sensitive Trading Framework for Real Financial Markets. arXiv:2511.12599.
- Lopez de Prado, M. 2016. Building Diversified Portfolios that Outperform Out of Sample. *The Journal of Portfolio Management*, 42(4): 59–69.
- Markowitz, H. M. 1952. Portfolio Selection. *The Journal of Finance*, 7(1): 77–91.
- Qu, B.; and Chen, M. 2026. CLQT: A Closed-Loop, Cost-Aware, Strategy-Consistent Benchmark for Diagnostic Evaluation of LLM Portfolio-Management Agents. arXiv:2606.29771.
- Rashid, M.; Hong, L.; Ding, J.; and Hossain, K. S. M. T. 2026. Fin-Analyst at FinMMEval 2026 Task 3: A Live Hybrid Trading Agent with LLM Specialists and Rule-Based Signals. arXiv:2607.12233.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.
- Tian, F.; Salim, F. D.; and Xue, H. 2025. TradingGroup: A Multi-Agent Trading System with Self-Reflection and Data-Synthesis. arXiv:2508.17565.
- Wang, S.; Yuan, H.; Ni, L. M.; and Guo, J. 2024. QuantAgent: Seeking Holy Grail in Trading by Self-Improving Large Language Model. arXiv:2402.03755.
- Xiao, Y.; Sun, E.; Luo, D.; and Wang, W. 2024. TradingAgents: Multi-Agents LLM Financial Trading Framework. arXiv:2412.20138.
- Yao, J.; and Zheng, Z. 2026. Beyond Agent Architecture: Execution Assumptions and Reproducibility in LLM-Based Trading Systems. arXiv:2606.08285.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- Yu, H.; Li, F.; and You, J. 2025. LiveTradeBench: Seeking Real-World Alpha with Large Language Models. arXiv:2511.03628.
- Yu, Y.; Li, H.; Chen, Z.; Jiang, Y.; Li, Y.; Zhang, D.; Liu, R.; Suchow, J. W.; and Khashanah, K. 2023. FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design. arXiv:2311.13743.
- Yu, Y.; Yao, Z.; Li, H.; Deng, Z.; Cao, Y.; Chen, Z.; Suchow, J. W.; Liu, R.; Cui, Z.; Xu, Z.; Zhang, D.; Subbalakshmi, K.; Xiong, G.; He, Y.; Huang, J.; Li, D.; and Xie, Q. 2024. FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. *arXiv preprint arXiv:2407.06567*.
- Zhang, W.; Zhao, L.; Xia, H.; Sun, S.; Sun, J.; Qin, M.; Li, X.; Zhao, Y.; Zhao, Y.; Cai, X.; Zheng, L.; Wang, X.; and An, B. 2024. A Multimodal Foundation Agent for Financial Trading: Tool-Augmented, Diversified, and Generalist. In *Proceedings of the 30th ACM SIGKDD Conference on*

Knowledge Discovery and Data Mining (KDD '24), 4314–4325. Barcelona, Spain.

Zhang, Y.; Goel, D.; Ahmad, H.; and Szabo, C. 2025. RegimeFolio: A Regime Aware ML System for Sectoral Portfolio Optimization in Dynamic Markets. arXiv:2510.14986.

Zhu, T.; Zhao, W.; Sun, R.; Luan, B.; Lu, J.; Wang, S.; Li, J.; Jiang, D.; He, Y.; and Bai, Z. 2026. From Knowing to Doing: A Memory-Controlled Benchmark for LLM Trading Agents on Stock Markets. arXiv:2605.28359.

Technical Appendix

This appendix is a self-contained technical supplement to *DarwinTrade: Hierarchical Self-Evolution for Multi-Agent Portfolio Trading*. Section and equation numbers of the form “Section ” or “Eq. 2” refer to the main paper. The appendix provides material that the eight-page main text references but cannot reproduce in full: the complete control-loop pseudocode (Appendix A), the verbatim LLM prompts and output schemas for every role and every evolution loop (Appendix B), implementation and hyperparameter details (Appendix C), the deterministic allocator specification (Appendix D), extended experimental protocol (Appendix E), the complete 2^3 factorial table (Appendix F), and the raw log excerpt behind the auditable case study (Appendix H). None of this material introduces claims beyond those in the main paper. It exists so that each reported result and mechanism can be independently reconstructed.

A Control-Loop Pseudocode

Algorithm 1 states the per-bar decision and delayed-feedback loop that the main text summarizes in Section , and Algorithm 2 states the periodic strategic-review loop of Section . The two are separated because they run on different clocks: the per-bar loop executes every trading day, whereas the strategic loop executes once every $N = 3$ completed episodes. Both loops share the strict one-bar delay: any realized outcome y_{t+1} is attached to the stored decision trace τ_t only after the day- $t+1$ opening auction, so it can never influence the decision that produced it. Lines that write to an evolution state $(\mathcal{U}^A, \mathcal{U}^T, \pi)$ are the only places where feedback re-enters the system. Execution (order generation and fills) is never modified by any LLM call.

B LLM Prompts and Output Schemas

DarwinTrade uses the LLM only inside clearly delimited modules: one regime classifier, four research analysts, and the reflection/evolution authors for the tactical and strategic loops. Every module is invoked with a fixed role instruction and is required to return strict JSON that matches a declared schema; malformed output triggers a deterministic fallback, not a free-form action. This appendix reproduces each instruction and schema verbatim from the released implementation, with only whitespace reflowed to fit the column. The prompts are grouped into research roles (Appendix B.1) and evolution authors (Appendix B.2).

All modules share the decoding configuration in Table 4: greedy decoding with a fixed seed and provider-side JSON-schema enforcement, so that a run is reproducible on any backend that honors the seed. When a provider rejects the `json_schema` response format, the client falls back to `json_object` mode and appends the schema to the system prompt, preserving the same output contract.

The reported backbone `mimo-v2.5-pro` is a proprietary, OpenAI-compatible chat model served behind a standard chat-completions endpoint. DarwinTrade issues only text prompts and reads back the JSON contract in Listing 3, so any instruction-following chat model with schema-constrained decoding, whether open-weight or proprietary,

Setting	Value
Backbone (reported runs)	<code>mimo-v2.5-pro</code> (OpenAI-compatible)
Temperature	0.0 (greedy)
Seed	1234 (fixed; sent when supported)
Response format	JSON schema, <code>json_object</code> fallback
Analyst score range	$x \in [-1, 1]$
Analyst confidence range	$c \in [0, 1]$
Regime labels	bull, bear, volatile

Table 4: LLM decoding and output configuration shared by all modules. The backbone is an interchangeable OpenAI-compatible chat model. DarwinTrade depends only on the JSON contract, not on a specific provider.

can be substituted without code changes. To support external replication we will release, under an anonymized repository, the full controller code, the fixed configuration of Table 5, the verbatim prompts and schemas of this appendix, and the per-day evidence cache and run logs behind every reported number, so a third party can reproduce the runs against frozen inputs or re-run them on an alternative backbone.

B.1 Research-Role Prompts

Each analyst receives the shared instruction skeleton in Listing 1 instantiated with a role-specific objective and constraint block (Listing 2), plus a longer role skill document elided here for space. All four analysts return the same four-field object in Listing 3; the regime classifier reuses the same skeleton with the objective in Listing 2. The resulting scores and confidences feed Eq. 1 unchanged.

B.2 Tactical and Strategic Evolution Prompts

The four evolution authors turn realized outcomes into either a short-lived tactical guard or a validated strategic patch. Listings 4 and 5 give the tactical reflection and evolution prompts; Listings 6 and 7 give the strategic reflection and policy-author prompts. These are the system prompts exactly as shipped. The per-call user message is a JSON payload of the relevant traces, outcomes, and track record. Each author also receives a longer role skill document that is elided here.

While the guard content is authored by the tactical LLM, the decision of when a guard may be installed is bounded by explicit quantitative gates that the payload exposes and the prompt enforces. A portfolio `haircut` in $[0.4, 1.0]$ is the default de-risking action and is considered only after a negative realized portfolio-alpha day. The `reduce_only` action is reserved for a name whose held position moved against the book while its analyst view weakened, so exposure may be trimmed but not flipped. The harder `avoid` action requires direct, symbol-specific reflection evidence, not a single losing bar. All three are further conditioned on the installer’s own track record: if the trailing `post_install_return` of recent guards of the same type was negative, the prompt requires reverting to no-action, so a guard family that has been hurting the book cannot keep re-arming. Every guard carries a 1–3 bar TTL and expires automatically, bounding the cost of any single mistaken install to a few bars. These gates make the tactical layer auditable: each install is attributable to

Symbol / field	Value	Meaning
<i>Analyst evolution (Eq. 2)</i>		
capsule size	200	tuples kept per (ρ, k)
warmup N	20	obs. before IC is used
shrink denom. add	20	$n/(n+20)$
cold-start u_0	0.01	skill before warmup
buy / sell threshold	± 0.15	on aggregate score $s_{t,i}$
min cross-section pts	5	below which IC is skipped
<i>Tactical / strategic loops</i>		
guard TTL	1–3 bars	auto-expire horizon
haircut range	[0.4, 1.0]	target-size scale
strategic period N	3 episodes	review cadence
strategic lookahead	20 days	history window $\mathcal{H}^S$
signal gate τ	0.5	below: no diagnosis
eval horizon H	[1, 30] bars	patch trial length
loss threshold ε	[0.005, 0.15]	rollback magnitude
pending patches	1	exclusivity
<i>Default policy π_0 and costs</i>		
gross budget G_0	0.95	$\sum_i w_i \leq G$
per-name cap $w_{\max}$	1.0	safety net (off)
min confidence	0.35	entry gate
drawdown guard	0.05	default threshold
rebound entry	0.02	default threshold
initial capital	\$100,000	per window
commission	1 bp	per trade
slippage	2 bps	per trade
fill ratio	1.0	full fills
universe size	20	fixed large-cap basket

Table 5: Fixed hyperparameters, default strategic policy, and cost model. Values are taken directly from the released configuration, and ablations perturb one entry at a time.

a logged trigger (negative alpha, weakening view, or specific adverse evidence) and to its realized post-install outcome.

C Implementation and Hyperparameters

Table 5 lists every fixed hyperparameter and default policy value used in the reported runs. The upper block gives the analyst-evolution constants of Eq. 2; the middle block gives the tactical-guard and strategic-review constants. The lower block gives the default strategic policy π_0 and the transaction-cost model. Ablations in the main paper change exactly one of these at a time (or disable one evolution loop) and leave the rest fixed.

The aggregation of analyst signals follows the IC-shrinkage formulation of Section . A warm capsule with positive IC contributes a skill weight $\max(0, \text{IC}) \cdot n/(n+20)$, and the fusion weight $\alpha_{t,i,k}$ of Eq. 1 is this skill times reported confidence (linear in c). The position-conviction step then re-weights each role by that same $\alpha_{t,i,k}$, so confidence enters sizing quadratically. This c^2 form is the canonical default, and the design-choice analysis (Table 9) contrasts it against a linear-confidence variant. A capsule with non-positive IC is dropped from aggregation instead of sign-flipped, which the signed-IC row of the design-choice table shows to be the safer choice.

Mode	Weight rule	Needs history
conf_weighted	$\propto c \cdot (1 - \text{risk})$	no
equal_weight	flat over eligible	no
confidence_concentrated	top-3 by c , equal	no
risk_parity	$\propto 1/(\text{risk} + \epsilon)$	no
inverse_risk_weighted	$\propto c(1 - \text{risk})^2$	no
kelly_capped	$\propto c^2(1 - \text{risk})$, top-3	no
min_variance	$\propto \Sigma^{-1} \mathbf{1}$	yes
max_sharpe	$\propto \Sigma^{-1} \mu$	yes
hrp	hierarchical risk parity	yes

Table 6: The nine allocator modes selectable by the strategic policy. Covariance modes (min_variance, max_sharpe, hrp) require ≥ 20 per-asset returns and otherwise fall back to conf_weighted. Gross exposure is split between long and short legs by the confidence mass on each side. A nonzero target_beta shifts this split without reselecting names.

D Deterministic Allocator

The allocator is a pure function of the guarded signals, the current portfolio, and historical returns under the strategic policy (Eq. 4). It contains no LLM call and no learned parameters beyond the strategic policy fields. The strategic loop can select among its modes but cannot alter its code. Table 6 lists the nine supported modes and their fallbacks. The three covariance-based modes require at least 20 historical returns per asset and otherwise downgrade to confidence weighting, which keeps the allocator well-defined early in each window.

E Extended Experimental Protocol

This section expands the setup summarized in the main paper. The universe is the fixed 20-stock basket GS, MSFT, HD, V, SHW, CAT, MCD, UNH, AXP, AMGN, TRV, CRM, JPM, IBM, HON, BA, AMZN, AAPL, PG, and JNJ. The same basket is used in both windows and is not reselected using period-specific information, so cross-window differences cannot come from survivorship or re-selection. Each window is evaluated independently: all three evolution states start empty and adapt only from feedback observed within that window, so “the same controller” refers to frozen code and policy bounds whose evolution restarts per window, not to state carried across windows. Each window has 83 decision bars and 82 return intervals.

Evidence admission is strictly point-in-time: price bars, news, fundamentals, and social data are admitted only when dated $< t$, and prices are split- and dividend-adjusted following standard vendor conventions. Because the allocator consumes only relative cross-sectional signals, the ex-post nature of these adjustments does not alter the ordering of returns used at any decision bar. Daily OHLCV bars, technical indicators, and quarterly fundamentals are sourced from Polygon. The news and social analysts consume a company-tagged news bundle drawn from Finnhub and Polygon, filtered to items published before t . News items are deduplicated with a stable key that prefers an explicit article identifier, then the canonical URL, then the title–publication-date pair, so

repeated syndications of one story are collapsed to a single admitted item. All external responses are cached in a per-day on-disk store keyed by symbol and date, which fixes the evidence each analyst sees and makes a run reproducible against the frozen cache without re-querying the vendors. Analyst reliability is learned only from realized post-decision returns, and allocation stays in the deterministic controller. Weights are frozen before the opening auction and the day- t open is used only as the fill price, so no outcome can influence the decision that generated it.

We report cumulative return, maximum drawdown, and annualized Sharpe, Sortino, and information ratio (against SPY), plus trade count. For paired comparisons we use a moving-block bootstrap (Künsch 1989) with block length 5 and 5000 resamples, reporting the paired excess-return sum $\Delta_\Sigma = 100 \sum_t (r_t^{\text{ours}} - r_t^{\text{comp}})$ in percentage points and P_+ , the bootstrap probability that $\Delta_\Sigma > 0$. Because each window is short, we treat P_+ as directional evidence, not a significance test, and the main paper avoids any significance claim. Baselines (buy-and-hold, SMA/WMA crossover, Bollinger bands, ATR bands, ARIMA, XGBoost, TradingAgents, FinAgent) use the same universe, windows, and cost model. The two external LLM agents are warmed up on the same universe for half a year of history before each window.

Turnover and Cost Sensitivity. Over the 83-bar windows DarwinTrade executes 1039 (rising) and 911 (down) individual fills, roughly 11–12 per bar over 20 names, far below the 1700–2000 fills of the daily-signal ARIMA, XGBoost, and SMA/WMA baselines and in the range of the discrete rule and LLM-agent baselines. At the reported 1 bp commission and 2 bps slippage this turnover costs 2.7% (rising) and 3.3% (down) of initial capital, already deducted from the Table 2 returns. As a first-order robustness check we rescale this friction linearly on the same fills: doubling all costs lowers net return to about 9.4% (rising) and 7.6% (down), which still leads every baseline. This scaling reprices the executed fills and does not re-optimize the policy under higher costs.

Short-Side Frictions and Fills. The reported runs assume full fills at the day- t open and do not model stock-loan fees, financing, or hard-to-borrow constraints. Two properties of the setup bound how much this idealization can flatter the results. First, all twenty names are continuously listed, large-cap, and index-constituent, i.e. general collateral for which borrow is deep and typically priced at a few tens of basis points annualized, not special. Second, short exposure is capped by the $G=0.95$ gross budget and the per-name weight cap, and the long/short split is set by the cross-sectional confidence mass instead of a fixed short target, so the short leg is only a fraction of NAV. As an upper bound we charge a borrow fee on the full gross budget for the entire window: at a general-collateral 50 bp annualized rate this deducts under 0.2 percentage points over the ≈ 0.33 -year window, and even a stressed 300 bp rate deducts under 1 percentage point, which is smaller than the doubled-cost check above and insufficient to change the baseline ranking. Because the universe carries no known hard-to-borrow names in either window, we do not model locate failures. Extending DarwinTrade to small-cap or special-borrow universes, where these frictions

Trend-rule label	Predicted			
	Bull	Volatile	Bear	Recall
<i>Rising-Market Window</i> (agreement 0.86, $n=81$)				
Bull	29	5	0	0.85
Volatile	0	17	0	1.00
Bear	0	6	24	0.80
<i>Down-Market Window</i> (agreement 0.86, $n=49$)				
Bull	15	5	1	0.71
Volatile	0	6	0	1.00
Bear	0	1	21	0.95

Table 7: Regime confusion against the deterministic trend rule, on days with a decisive directional label. Rows are the trend-rule label; columns are the LLM prediction.

bind, would require an explicit borrow-availability feed and is a natural direction for deployment-oriented follow-up.

E.1 Regime-Classifer Evaluation

The three regimes are latent, so no external ground-truth label exists. We instead score the classifier against a deterministic trend rule computed strictly before t : bull or bear when the SPY 20-bar backbone exceeds $\pm 1\%$, and volatile otherwise. This measures how the classifier’s calls align with the trend evidence and, more importantly, how its errors distribute across states.

Table 7 reports the confusion matrix. Agreement is 0.86 in both windows. The structure of the errors matters more than the rate: every misclassification lands in the adjacent volatile turning-point state, with no bull-bear sign flip in the rising window and only one in the down window. Reported confidence is calibrated to this: mean confidence is 0.75 on correct calls versus 0.59 (rising) and 0.64 (down) on incorrect ones, so low-conviction days concentrate the errors.

Two mechanisms limit the downstream cost of the residual errors. First, the errors are conservative: because they collapse a directional call into volatile instead of flipping its sign, the allocator’s regime-selected mode moves toward the lower-gross, risk-parity end instead of reversing exposure. Second, misclassified days do realize worse outcomes (mean daily NAV return is +18.9 bp on agreement days versus −20.7 bp on disagreement days in the rising window, and +0.9 versus −23.5 bp in the down window), but this is precisely the local error the one-to three-bar tactical guard is designed to trim before it compounds, consistent with the tactical layer carrying the largest single-layer ablation effect (Table 3).

F Complete Rising-Market Factorial

Table 8 reports all eight cells of the 2^3 evolution factorial in the Rising-Market Window. The main paper (Table 3) shows the four contrasts most relevant to the layer-ablation discussion, and here we add the three single-layer-only cells and restate the full-system and no-evolution cells for completeness. The pattern is consistent with the main text: the full system dominates, removing any single layer degrades

Factorial cell (A/T/S)		Return	Sharpe
Full	(1/1/1)	0.121	2.07
–Analyst	(0/1/1)	0.015	0.32
–Tactical	(1/0/1)	–0.091	–1.82
–Strategic	(1/1/0)	–0.053	–0.67
Only analyst	(1/0/0)	–0.074	–0.95
Only tactical	(0/1/0)	–0.023	–0.13
Only strategic	(0/0/1)	–0.023	–0.23
–All evolution	(0/0/0)	0.080	0.94

Table 8: Complete 2^3 factorial over analyst (A), tactical (T), and strategic (S) evolution in the Rising-Market Window. A “1” enables the layer, a “0” disables it. Return is cumulative, and Sharpe is annualized. The four boldfaceable contrasts and the no-evolution baseline appear in Table 3 of the main text. The three “only-” rows are added here.

Design	Rising market		Down market	
	Ret. (%)	Sharpe	Ret. (%)	Sharpe
Full system	12.1	2.07	10.9	2.56
Day-level IC shrinkage	16.4	2.76	9.0	2.35
Signed IC	–3.7	–0.56	0.5	0.19
Linear confidence	7.0	1.09	3.9	1.03
Rule-based regime	9.3	1.73	3.0	0.97
Shadow-policy rollback	8.9	1.50	1.1	0.37

Table 9: Sensitivity of the reported system to five design decisions. Each row perturbs one design axis and reports the resulting cumulative return and annualized Sharpe under both evaluation windows.

risk-adjusted performance, and every single-layer-only variant has negative Sharpe. No layer is individually sufficient. The layers must operate on one another’s evolving inputs.

G Sensitivity to Design Alternatives

Table 9 reports a sensitivity analysis of the reported system against five design alternatives, each perturbing one aggregation or governance decision at a time. The full system is repeated in the first row for reference. Each row measures the resulting return and Sharpe under both evaluation windows: day-level shrinkage yields a different bias-variance trade-off between the two windows, signed IC and linear confidence weaken both, a rule-based regime remains profitable but weakens under shift, and shadow-policy rollback trades return for the governance guarantee of reverting an injected harmful patch.

H Auditable Case Study: Full Log Excerpt

Table 10 reproduces the field-level records behind the AMGN case study of the main paper, recovered directly from the run logs. Each row is one logged evolution event, and the columns report only quantities present in the stored trace, so the narrative in the main text can be checked against the raw log. The three events act on distinct timescales, i.e., a reliability-driven analyst entry, a two-bar tactical guard, and

Layer	Date	Logged fields (verbatim from trace)
Analyst	entry	IC-weighted long, weight 5.7%, ref \$365.73, conf 0.43; closed +4 bars later after +4.1%; realized +\$252.78
Tactical	Feb 12	reduce-only on AMGN, haircut 0.80, TTL 2 bars; trimmed short from –22.3% to –7.9%; realized +\$499.23; expired before the Feb 13–17 rebound
Strategic	Mar 17	patch: optimizer $\rightarrow$ min_variance, position_size_multiplier = 0.85; rollback horizon + threshold logged; short sized down, closed after –2.3% for +\$390.47
Summary	window	post-patch realized P&L through Mar 31 = +\$366.30; symbol-total over the window = +\$3.07k

Table 10: Field-level log excerpt behind the AMGN case study (Figure 5). All values are recovered from the run logs, and no quantity is reconstructed after the fact.

a validated strategic patch, and each is attributable to a single named loop, not to one opaque reflection step.

Algorithm 1 DarwinTrade per-bar decision and delayed feedback

Input: trade date t ; universe $\mathcal{I}$; analyst roles $\mathcal{K}$; analyst reliability $\{u_{\rho,k}\}$; tactical state $\mathcal{U}_t^T$; strategic policy π_t ; portfolio P_t

Output: orders for day t ; updated evolution states

```

1: Close previous bar: attach outcome, then evolve
2: if  $t > t_{\text{start}}$  then
3:    $y_t \leftarrow$  open-to-open return over  $(t-1, t)$  {unavailable at  $t-1$ }
4:   attach  $y_t$  to stored trace  $\tau_{t-1}$ 
5:   append  $(x_{t-1,i,k}, c_{t-1,i,k}, y_{t,i})$  to capsule  $(\rho_{t-1}, k)$ 
6:   refresh  $u_{\rho_{t-1},k}$  via Eq. 2
7:    $q^T \leftarrow f_{\text{tac-reflect}}(\tau_{t-1}, y_t^{\text{pf}}, \mathcal{U}_t^T)$ 
8:    $\eta^T \leftarrow f_{\text{tac-evolve}}(q^T, \mathcal{H}^T, \mathcal{Z}^T, \pi_t)$ 
9:    $\mathcal{U}_t^T \leftarrow \text{INSTALL}(\mathcal{U}_t^T, \eta^T)$  {guards live 1–3 bars}
10: end if
11: Classify regime (evidence dated  $< t$ )
12:  $(\rho_t, \text{conf}) \leftarrow f_{\text{regime}}(\text{SPY}_{<t})$ 
13: if malformed then use deterministic trend rule
14: Multi-analyst research, parallel per asset
15: for all  $i \in \mathcal{I}, k \in \mathcal{K}$  do
16:    $(x_{t,i,k}, c_{t,i,k}) \leftarrow f_k(\text{evidence}_{i,<t})$ ; store in  $\tau_t$ 
17: end for
18: Regime-conditioned fusion (Eq. 1)
19: for all  $i \in \mathcal{I}$  do
20:    $\alpha_{t,i,k} \leftarrow u_{\rho_t,k} c_{t,i,k}$ 
21:    $s_{t,i} \leftarrow (\sum_k \alpha_{t,i,k} x_{t,i,k}) / (\sum_k \alpha_{t,i,k})$ 
22: end for
23:  $\mathcal{S}_t \leftarrow \{s_{t,i} : |s_{t,i}| > 0.15\}$  {buy if  $> 0.15$ , sell if  $< -0.15$ }
24: Apply tactical guards, then allocate
25:  $\tilde{\mathcal{S}}_t \leftarrow \text{APPLYGUARDS}(\mathcal{S}_t, \mathcal{U}_t^T)$ 
26:  $w_t \leftarrow \text{ALLOCATE}(\tilde{\mathcal{S}}_t, P_t, \mathcal{R}_{<t}; \pi_t)$ 
27:   s.t.  $|w_{t,i}| \leq w_{\text{max},t}, \sum_i |w_{t,i}| \leq G_t$ 
28: Freeze weights; day- $t$  open is fill price only
29: emit orders realizing  $w_t$ ; store decision trace  $\tau_t$ 
30: return orders,  $\{u_{\rho,k}\}, \mathcal{U}_t^T, \pi_t$ 

```

Algorithm 2 DarwinTrade periodic strategic review (every $N=3$ episodes)

Input: review index j ; policy π_j ; allowed parameter set Ω ; prior-patch outcomes $\mathcal{Z}_j^S$; pending patch p (or $\emptyset$)

Output: updated policy π_{j+1}

```

1: Evaluate the patch under trial before proposing a new one
2: if  $p \neq \emptyset$  then
3:    $g \leftarrow g_{m_p}(t_p, t)$  {cumulative alpha since install}
4:   if  $g < -\varepsilon_p$  then
5:      $\pi \leftarrow \text{REVERT}(p)$ ; log REVERTED;  $p \leftarrow \emptyset$ 
6:   else if  $t \geq t_p + H_p$  then
7:     log COMMITTED; record outcome in  $\mathcal{Z}^S$ ;  $p \leftarrow \emptyset$ 
8:   else
9:     return  $\pi_j$  {trial open; one patch at a time}
10:  end if
11: end if
12: Diagnose the multi-day pattern (20-day history)
13:  $z^S \leftarrow f_{\text{str-review}}(\mathcal{H}_j^S, \mathcal{Z}_j^S, \pi_j)$ 
14: if  $z^S.\text{signal} < \tau_{\text{gate}}$  then
15:   return  $\pi_j$  { $\tau_{\text{gate}}=0.5$ ; healthy}
16: end if
17: Author a minimal patch with a rollback trigger
18:  $(\Delta\pi, H, \varepsilon, m) \leftarrow f_{\text{str-policy}}(z^S, \pi_j, \Omega)$ 
19:  $\pi^+ \leftarrow \text{VALIDATE}(\pi_j + \Delta\pi; \Omega)$  {reject invalid}
20: if  $\pi^+$  valid then
21:   install  $\pi^+$  as pending patch  $p$  with  $(H, \varepsilon, m)$ , anchor  $t_p$ 
22:   return  $\pi^+$ 
23: end if
24: return  $\pi_j$ 

```

Listing 1 Shared role-instruction skeleton assembled by build_role_instruction.

```

Role: {title}
Objective: {objective}
Required outputs: {semicolon-separated
  required fields}
Constraints: {semicolon-separated
  constraints}
Tool order: {ordered tool list}
Report schema: {fields}
Input schema: {fields}
Output schema: {fields}
Success criteria: {criteria}
Skill instructions:
{role skill document}

```

Listing 2 Role objectives and key constraints for the regime classifier and the four research analysts (verbatim; skill bodies elided).

```
[market_regime_analyst] Market Regime
Analyst
Objective: Classify the current market
regime and describe how it
should constrain risk and execution.
Constraints: Use market structure
evidence only; return strict JSON
with summary, score, confidence,
market_regime.

[market_analyst] Market Analyst
Objective: Analyze trend, momentum,
support/resistance, and
volatility context.
Constraints: Use technical evidence only
; flag mixed signals.

[news_analyst] News Analyst
Objective: Analyze company and macro
news, timeliness, credibility,
and catalyst impact.
Constraints: Separate facts from rumor;
flag stale sources.

[social_analyst] Sentiment Analyst
Objective: Analyze the LEVEL and RATE-OF
-CHANGE of public/retail
sentiment as a SHORT-HORIZON
CONTRARIAN signal: extreme bullish
consensus after a run-up is a reversal
risk (score negative);
capitulation after a decline is a
bounce setup (score positive).
Constraints: Treat sentiment as mean-
reverting on a 1-5 day horizon,
not trend-following; weight the CHANGE
over the absolute level;
avoid rumor amplification.

[fundamentals_analyst] Fundamentals
Analyst
Objective: Analyze valuation, statements
, margins, leverage, and
cash flow quality.
Constraints: Anchor conclusions in
metrics; flag stale filings.
```

Listing 3 Shared analyst output schema. All four analysts and the regime classifier return this object, and malformed output falls back to a deterministic trend rule.

```
{
  "summary":      string,           //
                  evidence-grounded rationale
  "score":        number in [-1,1], //
                  directional view
  "confidence":   number in [0,1],  //
                  self-reported reliability
  "market_regime": "bull"|"bear"|"volatile"
}
```

Listing 4 Tactical reflection system prompt ($f_{\text{tac-reflect}}$).

```
You are DarwinTrade's tactical reflection
agent.
Look at YESTERDAY's trading day in
isolation. Identify the dominant
mistake (or say 'none'), and extract 1-3
lessons for tomorrow.
Do NOT propose strategy parameter changes
-- that is the strategic
layer's job.

CRITICAL -- distinguish "did not trade"
from "traded badly":
- If did_trade is False and orders_placed
is empty, the system held
its prior state. alpha=0 in that case is
EXPECTED, not a
wrong_direction signal. Use mistake_type
="none" or "missed_signal"
(only if a clear opportunity was ignored
), never "wrong_direction"
or "over_exposure" on a no-trade day.
- "wrong_direction" requires actual orders
to have been placed AND
alpha to be meaningfully negative.
- "over_exposure" requires real positions
AND a drawdown driven by
them.

Return valid JSON only.

# Output schema
{ "mistake_type": one of {over_exposure,
wrong_direction,
missed_signal, execution_friction,
risk_event, none},
"immediate_lessons": [string, ...], //
up to 3
"summary": string }
```

Listing 5 Tactical evolution system prompt ($f_{\text{tac-evolve}}$). Installs avoid / reduce-only / haircut guards with a 1–3 bar time-to-live.

You are DarwinTrade's tactical evolution agent.
Given yesterday's reflection, recent 5-day summary, AND your own past tactical-influence track record, decide whether to install short-term constraints for tomorrow. You are NOT limited to "tighten" -- you can also return an empty influence (no adjustment) when the right call is to do nothing.

The input includes 'track_record': each prior install_log entry with post_install_return (NAV % change while the influence was active or just after). USE IT:

- If your last few haircut/avoid installs had NEGATIVE post_install_return, they HURT the system. Be much more conservative -- prefer no influence over installing one that is likely to hurt again.
- If your last installs had POSITIVE post_install_return, the direction worked; you can install similar adjustments.
- Repeated 'wrong_direction' reflections do NOT automatically warrant a haircut. Check whether haircuts have actually helped recently before installing another one.
- An influence with avoid_symbols can lock the system out of a name during a reversal -- only avoid a symbol when reflection has direct, specific evidence (not just "lost money on it once").

Do NOT change strategy config. Return valid JSON only.

```
# Guard state installed (schema)
{ "avoid_symbols": [ticker, ...],          //
  "reduce_only_symbols": [ticker, ...],    //
  "portfolio_haircut": number in [0.4,    //
    1.0], // scales target sizes
  "ttl_days": integer in [1, 3] }          //
    auto-expire
```

Listing 6 Strategic reflection system prompt ($f_{\text{str-review}}$).

You are DarwinTrade's strategic reflection agent.
Look at the multi-day pattern across the lookback window. Identify ONE dominant cross-day pattern. Output strategic_signal_strength 0-1 -- high only if a strategy patch is genuinely warranted. Be conservative.

The input includes two critical feedback channels -- USE THEM:

1. 'objective_metrics': dd_from_peak, rolling_10d_return, trade_engagement_ratio_10d, current_nav, peak_nav.
 - A patch is rarely warranted when dd_from_peak > -0.02 AND rolling_10d_return > -0.01 -- the system is healthy, leave it.
 - Low trade_engagement_ratio_10d (< 0.3) signals under-deployment; consider RELAXING constraints, not tightening.
2. 'patch_track_record': every recent patch with status (pending / committed / reverted) and post_patch_return.
 - If a recent patch was reverted, do NOT propose it again.
 - If a recent patch committed with positive return, do not reverse it without strong reason.

CRITICAL -- when the system is already not trading, do NOT tighten min_confidence_threshold, max_single_position, max_gross_exposure, or position_size_multiplier; that makes a no-trade system trade even less. The correct response is to RELAX or do nothing.

Return valid JSON only.

```
# Output schema
{ "dominant_pattern": one of {alpha_decay,
  regime_shift,
  capsule_unstable, consistent_winner,
  drawdown_structural, none},
  "persistent_winners": [ticker, ...],
  "persistent_losers": [ticker, ...],
  "regime_observation": string,
  "strategic_signal_strength": number in
    [0,1],
  "summary": string }
```

Listing 7 Strategic policy-author system prompt ($f_{\text{str-policy}}$).
Emits a minimal patch over Ω plus a machine-executable
rollback trigger.

```
You are DarwinTrade's strategic policy
author.
Given a diagnosis, propose the MINIMAL
config patch needed. Use only
allowed fields. Smaller is better.

You MUST include a machine-executable
rollback_trigger with three
fields:
- eval_horizonBars: integer in [1, 30].
  How many bars the patch is
  held before it is committed (kept) or
  reverted.
- drawdown_pct: fraction in [0.005,
  0.15].
- reference: the monitored signal
  against which the loss threshold
  is applied.
The evaluator reads these fields directly;
free-text is ignored.

You see the outcome of your own PAST
patches in patch_track_record
(status + nav_impact). If recent patches
were reverted, propose a
different direction; blindly tightening
again is not self-evolution.

DIRECTIONAL CONTROL -- target_beta in [-1,
1] is your primary lever
over net exposure (the code holds no
directional prior). It shifts the
long/short budget split WITHOUT changing
which symbols are selected.
Escalate a directional bet while a trend
AND your prior patch's
positive alpha both persist; lean back
toward 0 when momentum turns.

Return valid JSON only.

# Allowed patch fields (Omega)
max_gross_exposure      in [0.3, 1.0]
max_single_position      in [0.05, 1.0]
min_confidence_threshold in [0.0, 0.6]
position_size_multiplier in [0.5, 3.0]
drawdown_guard_threshold in [0, 1]
rebound_entry_threshold  in [0, 1]
preferred_optimizer      in {
  conf_weighted, equal_weight,
  confidence_concentrated, risk_parity,
  inverse_risk_weighted,
  kelly_capped, min_variance,
  max_sharpe, hrp}
target_beta              in [-1, 1]
# Any unknown field or out-of-range value
is rejected by Validate().
```
